Project 1 - Description

1. Overview

This lab focuses on the **thread subsystem** in Pintos. You will extend the baseline kernel to support:

- an efficient sleep/wakeup mechanism (Alarm Clock),
- **priority scheduling** (including preemption),
- **priority donation** to address priority inversion,

Your work is mainly in `src/threads/` and partially in `src/devices/`.

---

2. Learning Objectives

By the end of this lab, you should be able to:

1. Explain how Pintos threads transition among **running / ready / blocked** states.
2. Implement scheduling policies that depend on thread priority.
3. Use synchronization primitives correctly under concurrency and interrupt contexts.
4. Understand priority inversion and how **priority donation** mitigates it.
5. (Advanced) Understand the idea of a multi-level feedback queue scheduler and fixed-point arithmetic.

---

3. Repository Layout (Where You Work)

- `src/threads/`
  Primary implementation for thread management and scheduling.
- `src/devices/`
  Timer-related logic (used by Alarm Clock).
- `src/tests/`
  Test programs and check scripts.
  Note: You may modify tests for local debugging, but official grading typically uses the original tests.

---

4. Background

4.1 Understanding Threads

The first step is to read and understand the code for the initial thread system.

Pintos already implements thread creation and thread completion, a simple scheduler to switch between threads, and synchronization primitives (semaphores, locks, condition variables, and optimization barriers).

Some of this code might seem slightly mysterious. You can read through parts of the source code to see what's going on.

If you like, you can add calls to printf() almost anywhere, then recompile and run to see what happens and in what order.

4.2 Create a New Thread

When a thread is created, you are creating a new context to be scheduled. You provide a function to be run in this context as an argument to `thread_create()`.

The first time the thread is scheduled and runs, it starts from the beginning of that function and executes in that context.
When the function returns, the thread terminates. Each thread, therefore, acts like a mini-program running inside Pintos, with the function passed to thread_create() acting like main().

At any given time, exactly one thread runs and the rest, if any, become inactive. The scheduler decides which thread to run next.

If no thread is ready to run at any given time, then the special "idle" thread, implemented in idle(), runs.

Synchronization primitives can force context switches when one thread needs to wait for another thread to do something.

4.3 Context Switch

The mechanics of a context switch are in threads/switch.S, which is 80x86 assembly code. (You don't have to understand it.)

It saves the state of the currently running thread and restores the state of the thread we're switching to.

In Pintos, each thread is assigned a small, fixed-size execution stack just under 4 kB in size.

The kernel tries to detect stack overflow, but it cannot do so perfectly. You may cause bizarre problems, such as mysterious kernel panics, if you declare large data structures as non-static local variables, e.g. int buf[1000];.

4.4 Explanation

This part provides an overview of the important functions related to lab1.

How the Timer Interrupt Works

1. Setup (The Registration)

- `timer_init()`: Sets the *hardware timer* frequency and registers `timer_interrupt()` as the handler.
- The Interrupt Switch: `intr_enable()` and `intr_disable()`
  - `intr_enable()` **(The "ON" Switch)**: This allows the CPU to "hear" the hardware timer. It means the timer is allowed to interrupt your program at any moment to let another thread run.
  - `intr_disable()` **(The "OFF" Switch)**: This tells the CPU to "ignore" all external signals. Your current code will run from start to finish without being interrupted.
  - You use these when you are in a "Critical Section"—a part of the code where you are modifying shared data (like the `ready_list` or a file table).
  - Code Example:

```
/* 1. Disable interrupts and save the previous state */

enum intr_level old_level = intr_disable ();

/* --- Start of Critical Section --- */

/* Perform operations that must not be interrupted here */

/* --- End of Critical Section --- */

/* 2. Restore interrupts to the previous state */

intr_set_level (old_level);
```

2. Execution (The Tick)

- Hardware Timer: Pulses every 10ms (at 100Hz).
- `timer_interrupt()`: Increments the tick count and calls `thread_tick()`.
- `thread_tick()`: Increments thread statistics. If the time slice is exhausted, it calls `intr_yield_on_return()` to set a **yield flag**.

3. Interrupt Epilogue (The Safe Exit)

- `intr_handler()`: Once the specific timer logic is done, the main handler checks the yield flag.

- If `yield_on_return == true`, it finally calls `thread_yield()`.
- This step is crucial because it ensures the CPU has finished all hardware-related "cleanup" before attempting to switch the thread's context.

4. Transition (The Switch)

- `thread_yield()`: Moves the current thread from Running to Ready and puts it back in the `ready_list`.
- `schedule()`: Decides which thread runs next.
- `switch_threads()`: The "magic" assembly function that swaps CPU stack pointers. Once the stack is swapped, the thread has changed.
- `intr_exit` (Assembly): When the thread eventually wakes up again, it finishes the assembly routine to restore registers and return to its original code.

Tools

- kernel/list.c
- kernel/list.h

Doubly linked list implementation.
Used all over the Pintos code, and you'll probably want to use it a few places yourself in project 1.

4.5 Synchronization

Proper synchronization is an important part of the solutions to these problems.

Any synchronization problem can be easily solved by turning interrupts off: while interrupts are off, there is no concurrency, so there's no possibility for race conditions. Therefore, it's tempting to solve all synchronization problems this way, but don't.

Instead, use **semaphores, locks, and condition variables** to solve the bulk of your synchronization problems.
Read the comments in **threads/synch.c** if you're unsure what synchronization primitives may be used in what situations.

In the Pintos projects, the only class of problem best solved by disabling interrupts is coordinating data _shared between a kernel thread and an interrupt handler_.

Because interrupt handlers can't sleep, they can't acquire locks. This means that data shared between kernel threads and an interrupt handler must be protected within a kernel thread by turning off interrupts.

This project only requires accessing a little bit of thread state from interrupt handlers. For the alarm clock, the timer interrupt needs to wake up sleeping threads.

When you do turn off interrupts, take care to do so for the least amount of code possible. Otherwise you can end up losing important things such as timer ticks or input events.
Turning off interrupts also increases the interrupt handling latency, which can make a machine feel sluggish if taken too far.

The synchronization primitives themselves in **synch.c** are implemented by disabling interrupts.
You may need to increase the amount of code that runs with interrupts disabled here, but you should still try to keep it to a minimum.

Disabling interrupts can be useful for debugging, if you want to make sure that a section of code is not interrupted.

You should remove debugging code before turning in your project. (Don't just comment it out, because that can make the code difficult to read.)

5. Tasks and Requirements

5.1 Task 1 — Alarm Clock

Goal: Replace busy waiting with a blocking sleep mechanism.

Required behavior:

- `timer_sleep(ticks)` blocks the current thread for *at least* `ticks` timer ticks.
- Threads should not waste CPU by spinning while waiting.
- When multiple threads sleep, wakeups should occur correctly and efficiently.

What to think about (no solution here):

- Where to store sleeping threads and their wake-up time.
- How to safely access shared structures when interrupts may occur.

Step 1: Add a Wakeup Timestamp to the Thread

A thread needs to know its deadline.

Step 2: Create a Global "Sleep Queue"

You need a place to track threads that aren't ready to run. Moving these to a separate list removes them from the scheduler's pool, ensuring they don't waste CPU cycles while waiting.

Step 3: Block the Thread in timer_sleep()

Replace the yield loop with a formal 'clock out.' This pulls the thread from the scheduler's pool immediately, ensuring it doesn't waste cycles checking its own alarm.

Step 4: Update timer_interrupt()

The timer interrupt now acts as the "wake-up call" for the system.

5.2 Task 2 — Priority Scheduling

Goal: The scheduler must always choose the highest-priority ready thread.

Required behavior:

- If a higher-priority thread becomes ready, the CPU should be **preempted immediately** (do not wait for the next timer tick).
- Synchronization primitives must respect priority:
    - When unblocking a thread (e.g., semaphore/lock/condition), unblock the **highest-priority waiter** first.

Priority ranges:

- Priority is in `[PRI_MIN, PRI_MAX]` with a default of `PRI_DEFAULT`.
  (Do not change these constants.)

Step1. Highest Priority Always Runs

Instead of following a "first-come, first-served" rule, always wake the thread with the highest priority from any waiting queue. Whether a thread is waking up, the system must ensure that the most urgent task is moved back to the `ready_list` first.

5.3 Task 3 — Priority Donation

Goal: Prevent priority inversion when a high-priority thread waits on a lock held by a lower-priority thread.

Required behavior:

- When a high-priority thread waits for a lock, it donates its priority to the lock holder.
- Donation must support:
    - Multiple donors
    - Nested donation (a donation chain)
- When a lock is released, the holder’s effective priority should update correctly.

Step 1. Extended Thread State

Add fields to each thread to track its **original priority**, a **collection of donors**, and the **specific lock** it is currently waiting for. This data provides the kernel with the necessary "memory" to manage temporary priority boosts and resolve dependencies.

Step 2. Proper Initialization

Ensure these fields are correctly initialized during thread creation: set the original priority to the current value, and leave the donor list and waiting lock empty. This establishes a clean, consistent baseline for every new task entering the system.

Step 3. Priority Propagation

When a thread blocks due to an unavailable lock, it must register as a donor to the current lock-holder. This priority boost must propagate upward through the chain of holders (with a defined depth limit 8) to ensure the entire bottleneck is cleared as quickly as possible.

Step 4. Acquisition Update

When a thread successfully acquires a lock, clear its recorded waiting lock and officially become the new holder of that lock.

Step 5. Recalculation on Release

When releasing a lock, remove all donors associated with that specific lock and recalculate the thread's effective priority. The new priority should be the maximum of its original priority and the highest remaining donation from any other locks it still holds.

Step 6. System-Wide Enforcement

Maintain all ready and synchronization queues in a strictly sorted order based on the effective priority. Whenever a priority changes or a thread is unblocked, perform an immediate preemption check to guarantee that the highest-priority thread in the system is always the one running.

5.4 Task 4: Implement the following functions

Implement the following functions that allow a thread to examine and modify its own priority. Skeletons for these functions are provided in threads/thread.c.

`void thread_set_priority (int new_priority)`

Sets the current thread's priority to new_priority.
If the current thread no longer has the highest priority, yields.

`int thread_get_priority (void)`

Returns the current thread's priority. In the presence of priority donation, returns the higher (donated) priority.

---

6. Suggested Workflow (Recommended Implementation Order)

1. Implement Alarm Clock and verify sleep-related tests.
2. Implement basic priority scheduling (ready list ordering + preemption).
3. Update synchronization primitives to wake the highest-priority waiters.
4. Implement priority donation and verify inversion-related tests.

---

7. Build & Run: Command Cheat Sheet

7.1 Run all tests

Each project has several tests. To completely test your implementation, invoke `make check` from the project build directory.

- This will build and run each test and print a "pass" or "fail" message for each one.
- When a test fails, `make check` also prints some details of the reason for failure.
- After running all the tests, make check also prints a summary of the test results.

To run only the tests for Project 1, run `make check` in the `src/thread` directory.

7.2 Run an individual test

You can also run an individual test at a time.

e.g. A given test `foo` writes its output to `foo.output`, then a script scores the output as "pass" or "fail" and writes the result to `foo.result`.

To run and grade a single test, make the `.result` file explicitly from the build directory.

e.g. Type `make tests/threads/alarm-multiple.result` in the build directory to test the alarm-multiple case.

If make says that the test result is up-to-date, but you want to re-run it anyway

- either delete the `.output` file by hand, e.g. `rm tests/threads/alarm-multiple.output`
- or run `make clean` to delete all build and test output files.

7.3 Result

The first time you run it, you'll see:

```
Windows PowerShell
+ (priority-donate-chain) interloper 4 finished.
+ (priority-donate-chain) main should have priority 15.  Actual priority: 0.
+ (priority-donate-chain) interloper 5 finished.
+ (priority-donate-chain) main should have priority 18.  Actual priority: 0.
+ (priority-donate-chain) interloper 6 finished.
+ (priority-donate-chain) main should have priority 21.  Actual priority: 0.
  (priority-donate-chain) main finishing with priority 0.
  (priority-donate-chain) end
pass tests/threads/alarm-single
pass tests/threads/alarm-multiple
pass tests/threads/alarm-simultaneous
FAIL tests/threads/alarm-priority
pass tests/threads/alarm-zero
pass tests/threads/alarm-negative
FAIL tests/threads/priority-change
FAIL tests/threads/priority-donate-one
FAIL tests/threads/priority-donate-multiple
FAIL tests/threads/priority-donate-multiple2
FAIL tests/threads/priority-donate-nest
FAIL tests/threads/priority-donate-sema
FAIL tests/threads/priority-donate-lower
FAIL tests/threads/priority-fifo
FAIL tests/threads/priority-preempt
FAIL tests/threads/priority-sema
FAIL tests/threads/priority-condvar
FAIL tests/threads/priority-donate-chain
13 of 18 tests failed.
../../tests/Make.tests:23: recipe for target 'check' failed
make: *** [check] Error 1
root@99cc7e840e06:~/pintos/src/threads/build#
```

Now it's your time to shine!🎉🎉🎉
Once you've completed all the tasks, you will see:

```
Windows PowerShell
pintos -v -k -T 60 --bochs  -- -q  run priority-condvar < /dev/null 2> tests/threads/priority-condvar.errors > tests/thr
eads/priority-condvar.output
perl -I../.. ../../tests/threads/priority-condvar.ck tests/threads/priority-condvar tests/threads/priority-condvar.resul
t
pass tests/threads/priority-condvar
pintos -v -k -T 60 --bochs  -- -q  run priority-donate-chain < /dev/null 2> tests/threads/priority-donate-chain.errors >
 tests/threads/priority-donate-chain.output
perl -I../.. ../../tests/threads/priority-donate-chain.ck tests/threads/priority-donate-chain tests/threads/priority-don
ate-chain.result
pass tests/threads/priority-donate-chain
pass tests/threads/alarm-single
pass tests/threads/alarm-multiple
pass tests/threads/alarm-simultaneous
pass tests/threads/alarm-priority
pass tests/threads/alarm-zero
pass tests/threads/alarm-negative
pass tests/threads/priority-change
pass tests/threads/priority-donate-one
pass tests/threads/priority-donate-multiple
pass tests/threads/priority-donate-multiple2
pass tests/threads/priority-donate-nest
pass tests/threads/priority-donate-sema
pass tests/threads/priority-donate-lower
pass tests/threads/priority-fifo
pass tests/threads/priority-preempt
pass tests/threads/priority-sema
pass tests/threads/priority-condvar
pass tests/threads/priority-donate-chain
All 18 tests passed.
root@dfc694dcca2c:~/pintos/src/threads/build#
```

Note: Whenever you modify your code, you **must** run the following command in your build directory:

```
make clean
```

If you skip this step, leftover artifacts from previous runs—specifically the `.result` and `.output` files—will persist. When you trigger the grading script again, it will count these "ghost" results alongside your new ones, causing the total number of test cases to increase incorrectly.

---

8. Common Pitfalls & Debugging Tips

- **Small kernel stacks:** Avoid large local arrays/structs in kernel code.
- **Preemption correctness:** If a higher-priority thread becomes ready, you must yield/preempt immediately.
- **Waiter ordering:** Ensure wait queues for locks/semaphores/conditions are priority-aware.
- **Donation bookkeeping:** Donation must be undone correctly when locks are released.
- **Reproducible debugging:** Run one test repeatedly until stable before moving on.

---

9. Deliverables (Placeholder)

- The entire pintos project
- A design document.
- Submission packaging method: **TBD** (to be updated once confirmed).

10. Grading

Participation: 4% for 4 lab sessions of Project 1.

Final codes: 6% (10% penalty for each day if submitted late). The scores for all test cases are the same. One can obtain the score of a test case by passing it.

Final report: 5% (10% penalty for each day if submitted late)

Submission: After completing the project, students are required to compress the Pintos code into a ZIP file and upload it to Canvas.

Regarding the report, please write comments for the code they modify or add for each line, explain their design, and provide detailed design and solution descriptions in the final report. The code and the final report should be consistent.